

Document d'Architecture Logicielle (SAD)

Loka House — Plateforme d'annonces immobilières vérifiées

Projet	Loka House
Version du document	1.0
Date	15 juin 2026
Statut	Brouillon pour validation
Type d'application	Application (Android/iOS) + Back-office web

1. Introduction

1.1 Objet du document

Ce document décrit l'architecture logicielle de **Loka House**, une plateforme de mise en relation entre propriétaires/agences immobilières et locataires, dont la valeur centrale est la **lutte contre les arnaques** (fausses annonces, faux commissionnaires, photos trompeuses).

Il sert de référence technique pour l'équipe de développement, les parties prenantes et la maintenance future.

1.2 Portée

- Loka House couvre :
- La publication d'annonces de location **vérifiées** (photo + vidéo obligatoires).
 - La recherche géolocalisée (commune / quartier) avec filtres.
 - La mise en relation directe locataire ↔ propriétaire.
 - La vérification d'identité (anti-arnaque).
 - La monétisation (annonces premium, abonnements agences, publicité, commission de visite).

1.3 Définitions et acronymes

Terme	Définition
-------	------------

Annonce	Offre de location publiée par un propriétaire ou une agence.
Bailleur	Propriétaire ou agence qui publie une annonce.
Locataire	Utilisateur qui recherche un logement.
KYC	<i>Know Your Customer</i> — vérification d'identité.
SAD	<i>Software Architecture Document</i> .
Commune / Quartier	Découpage administratif local (ex. RDC : commune puis quartier).
Visite vérifiée	Visite physique organisée et tracée via la plateforme.

1.4 Le problème adressé

Sur le marché actuel, chercher une maison à louer expose à :

- des **fausses annonces** (logements inexistants ou déjà loués) ;
- des **commissionnaires** abusifs qui facturent sans valeur ajoutée ;
- des **photos trompeuses** ne correspondant pas au bien réel.

Loka House répond par : **vérification obligatoire (photo + vidéo + ID), contact direct, et traçabilité des visites.**

2. Objectifs et contraintes d'architecture

2.1 Objectifs qualité (non fonctionnels)

Attribut	Cible
Fiabilité de la vérification	100 % des annonces publiées passent par un workflow de validation.
Performance	Recherche d'annonces < 1 s ; chargement liste < 2 s sur 3G.
Disponibilité	99,5 % (cible de démarrage).
Scalabilité	Démarrage mono-ville, montée en charge multi-villes sans refonte.
Sécurité	Données d'identité chiffrées, RGPD/loi locale sur les données personnelles.
Accessibilité réseau	Optimisé pour connexions lentes (compression média, mode léger).
Multilingue	Français (priorité 1), Lingala/Swahili (priorité 2), anglais (optionnel).

2.2 Contraintes

- **Contexte réseau** : bande passante limitée et coûteuse → médias compressés côté serveur, upload résilient.
- **Paiements** : intégration **Mobile Money** (Orange Money, M-Pesa, Airtel Money) prioritaire sur la carte bancaire.
- **Appareils cibles** : smartphones Android d'entrée/milieu de gamme majoritaires.
- **Vidéo obligatoire** : nécessite stockage et CDN optimisés en coût.

3. Acteurs et cas d'usage

3.1 Acteurs

Acteur	Rôle
Visiteur	Navigue et consulte les annonces sans compte.
Locataire	Recherche, contacte les bailleurs, planifie des visites.
Bailleur (propriétaire)	Publie et gère ses annonces.
Agence	Bailleur professionnel avec abonnement et annonces multiples.
Modérateur	Valide les annonces (vérification photo/vidéo/ID).
Administrateur	Gère utilisateurs, monétisation, statistiques, litiges.

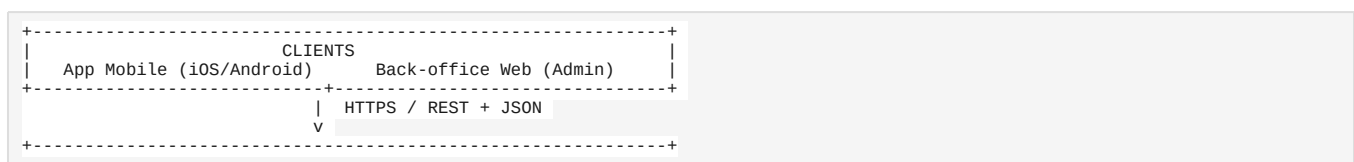
3.2 Cas d'usage principaux

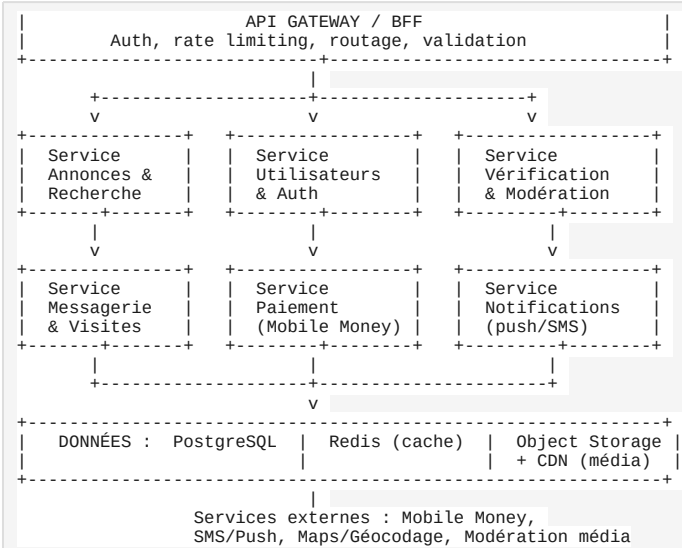
1. **Publier une annonce** → upload photos + vidéo obligatoires → soumission à modération → publication.
2. **Vérifier son identité (KYC)** → upload pièce d'identité → validation.
3. **Rechercher un logement** → filtres commune/quartier/prix/chambres → résultats.
4. **Contacter un bailleur** → messagerie in-app + appel masqué optionnel.
5. **Planifier une visite vérifiée** → réservation → suivi → paiement commission.
6. **Modérer une annonce** → revue média + ID → approuver/rejeter avec motif.
7. **Souscrire un service payant** → annonce premium / abonnement agence via Mobile Money.

4. Vue d'ensemble de l'architecture

4.1 Style architectural

Architecture **client-serveur** avec un **back-end modulaire (API REST)** organisé en services logiques. On démarre sur un **monolithe modulaire** (plus simple à opérer et moins coûteux au lancement) conçu pour pouvoir extraire des micro-services plus tard (paiement, média, notifications).





4.2 Vue logique (modules)

Module	Responsabilité
Auth & Utilisateurs	Inscription, connexion (OTP SMS), profils, rôles.
Vérification (KYC)	Upload et validation des pièces d'identité, statut de confiance.
Annonces	Création, édition, cycle de vie (brouillon → modération → publiée → archivée).
Média	Upload résilient, compression, génération de miniatures, watermark, CDN.
Recherche	Indexation géo + filtres (prix, chambres, commune, quartier).
Modération	File d'attente de validation, outils modérateur, motifs de rejet.
Messagerie & Visites	Chat in-app, planification et suivi des visites.
Paiement	Intégration Mobile Money, facturation, abonnements, commissions.
Notifications	Push, SMS, e-mail.
Back-office	Tableaux de bord admin, modération, monétisation, litiges.

5. Pile technologique recommandée

Recommandation par défaut, ajustable selon les compétences de l'équipe.

Couche	Choix recommandé	Justification
--------	------------------	---------------

App mobile	Flutter (Dart)	Une base de code iOS+Android, perfs natives, bon sur appareils d'entrée de gamme. (<i>Alternative : React Native.</i>)
Back-office web	React + TypeScript	Écosystème mature pour interfaces admin riches.
Back-end API	Node.js (NestJS, TypeScript) ou Python (Django/DRF)	Productivité, écosystème, structure modulaire.
Base de données	PostgreSQL (+ PostGIS pour la géo)	Relationnel robuste + recherche géospatiale native.
Cache / files	Redis	Cache de recherche, sessions, files de tâches.
Stockage média	Object storage (S3-compatible) + CDN	Coût maîtrisé, diffusion rapide des photos/vidéos.
Recherche	PostgreSQL/PostGIS au début → OpenSearch si volume élevé	Démarrer simple, scaler ensuite.
Auth	JWT (access + refresh) + OTP SMS	Standard mobile, sans mot de passe possible.
Notifications	Firebase Cloud Messaging + passerelle SMS locale	Push gratuit + SMS pour OTP/alertes.
Infra	Docker, déploiement cloud (VPS/managed)	Portable, reproductible.
CI/CD	GitHub Actions	Build/test/déploiement automatisés.

6. Modèle de données (principales entités)

```

User (id, téléphone, nom, rôle, kyc_statut, date_création, note_confiance)
■ ■ < Listing (annonce)
KYCDocument (id, user_id, type_pièce, url_chiffrée, statut, vérifié_par, date)
Listing (id, bailleur_id, titre, description, prix, devise, nb_chambres,
        type_bien, commune_id, quartier_id, géoloc(lat,lng),
        statut[brouillon|en_modération|publiée|rejetée|archivée],
        premium[bool], date_création, date_expiration)
■ ■ < Media (id, listing_id, type[photo|video], url, miniature, ordre, vérifié)
■ ■ < Visit (visite)
Location (commune_id, quartier_id, nom, parent_id) # référentiel géo
Conversation (id, listing_id, locataire_id, bailleur_id)
■ ■ < Message (id, conversation_id, expéditeur_id, contenu, date, lu)
Visit (id, listing_id, locataire_id, bailleur_id, date_prévue,
       statut[demandée|confirmée|effectuée|annulée], commission_due)
Payment (id, user_id, type[premium|abo_agence|commission|pub],
         montant, devise, fournisseur_mm, statut, ref_transaction, date)
Subscription (id, agence_id, plan, début, fin, statut)
ModerationLog (id, listing_id, modérateur_id, action, motif, date)
Report (id, signalé_par, listing_id, motif, statut) # signalement d'arnaque

```

Index clés : Listing(commune_id, quartier_id, prix, nb_chambres, statut), index géospatial PostGIS sur Listing.géoloc.

7. Conception des API (extrait REST)

Méthode	Endpoint	Description	Accès
POST	/auth/register	Inscription + envoi OTP	Public
POST	/auth/verify-otp	Validation OTP, retour JWT	Public
POST	/kyc/documents	Upload pièce d'identité	Authentifié
GET	/listings	Recherche + filtres (commune, quartier, prix_min/max, chambres)	Public
GET	/listings/{id}	Détail d'une annonce	Public
POST	/listings	Créer une annonce (brouillon)	Bailleur KYC OK
POST	/listings/{id}/media	Upload photo/vidéo	Bailleur
POST	/listings/{id}/submit	Soumettre à modération	Bailleur
POST	/moderation/{id}/approve	Approuver	Modérateur
POST	/moderation/{id}/reject	Rejeter (avec motif)	Modérateur
POST	/conversations	Démarrer une discussion	Locataire
POST	/visits	Planifier une visite	Locataire
POST	/payments/premium	Payer une annonce premium (Mobile Money)	Bailleur
POST	/payments/webhook	Callback fournisseur Mobile Money	Système
POST	/reports	Signaler une annonce suspecte	Authentifié

8. Architecture de sécurité et anti-arnaque

C'est le **cœur de différenciation** du produit.

8.1 Vérification d'identité (KYC)

- Upload obligatoire d'une pièce d'identité pour **publier** une annonce.
- Statut de confiance par utilisateur (non vérifié → en cours → vérifié).
- Badge « Bailleur vérifié » visible sur les annonces.
- Documents d'identité **chiffrés au repos**, accès restreint aux modérateurs.

8.2 Vérification des annonces

- **Photo + vidéo obligatoires** : pas de publication sans média réel.
- **Watermark** automatique + horodatage sur les médias pour limiter la réutilisation de photos volées.
- Détection des **doublons média** (hash perceptuel) pour repérer les photos recyclées entre annonces.
- File de **modération humaine** avant mise en ligne.

8.3 Protection des utilisateurs

- **Contact direct** propriétaire ↔ locataire (élimine l'intermédiaire/commissionnaire abusif).
- **Numéros masqués** en option (proxy d'appel) pour protéger la vie privée.
- **Système de signalement** + suspension automatique au-delà d'un seuil de signalements.
- **Notes et avis** sur les bailleurs après visite.
- **Traçabilité des visites** : visite organisée via l'appli, preuve d'existence du bien.

8.4 Sécurité technique

- HTTPS/TLS partout, JWT à durée courte + refresh tokens.
- Rate limiting et protection anti-brute-force sur l'auth.
- Validation/sanitisation des entrées, protection injection SQL (ORM paramétré).
- Journalisation des actions sensibles (modération, paiements).
- Conformité protection des données : minimisation, consentement, droit à l'effacement.

9. Module de paiement (Mobile Money)

Aspect	Détail
Fournisseurs cibles	Orange Money, M-Pesa (Vodacom), Airtel Money.
Flux	Initiation paiement → redirection/USSD/STK push → callback webhook → confirmation.
Idempotence	Référence de transaction unique pour éviter les doubles débits.
Réconciliation	Journal de transactions + statut, rapprochement automatique via webhook.
Sécurité	Vérification de signature des webhooks, jamais de logique de crédit côté client.

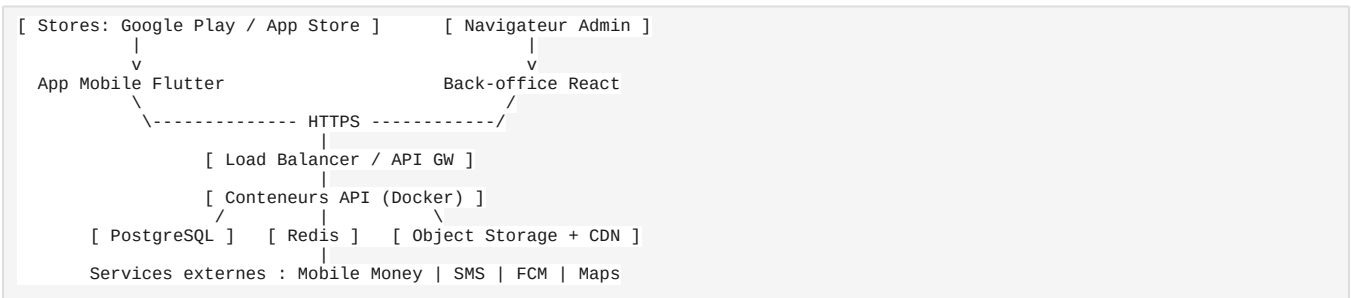
■■ **Important pour la commission de visite (1 mois de loyer)** : ce flux peut être perçu comme de l'intermédiation immobilière et est **réglementé** dans la plupart des pays. À cadrer avec un conseil juridique local et à formaliser dans des CGU claires (qui paie, quand, conditions de remboursement).

10. Modèle de monétisation (implémentation)

Source	Mécanisme technique
--------	---------------------

Annonce premium (5–20 \$)	Flag premium + mise en avant dans la recherche, durée limitée, paiement Mobile Money.
Abonnement agence (50–100 \$/mois)	Entité <code>Subscription</code> , plans, renouvellement, quota d'annonces.
Publicité in-app	Emplacements natifs entre les résultats, régie pub ou ventes directes.
Commission de visite (1 mois loyer)	Entité <code>Visit.commission_due</code> , déclenchée à la visite confirmée/effectuée.

11. Vue de déploiement



- **Environnements** : dev → staging → production.
- **CI/CD** : build, tests, scan sécurité, déploiement conteneurisé.
- **Sauvegardes** : base de données quotidienne + rétention, médias répliqués.
- **Observabilité** : logs centralisés, métriques, alertes (erreurs, latence, échecs paiement).

12. Phasage proposé (MVP → V2)

Phase	Contenu
MVP	Auth OTP, KYC bailleur, création annonce (photo+vidéo), modération, recherche commune/quartier + filtres, contact direct, signalement.
V1	Paiement Mobile Money, annonces premium, notifications push, avis bailleurs.
V2	Abonnements agences, planification/commission de visite, publicité, multi-villes, détection de doublons média avancée.

13. Risques et mitigations

Risque	Impact	Mitigation
--------	--------	------------

Coût stockage/diffusion vidéo	Élevé	Compression serveur, limite de durée vidéo, CDN économique.
Contournement de la vérification	Critique (réputation)	Modération humaine + détection doublons + signalements.
Cadre juridique de la commission de visite	Juridique	Validation avocat local, CGU explicites.
Faible connectivité utilisateurs	Adoption	Mode léger, upload résilient, images progressives.
Fraude au paiement Mobile Money	Financier	Webhooks signés, idempotence, réconciliation.

14. Décisions d'architecture (résumé)

1. **Monolithe modulaire d'abord**, micro-services plus tard → simplicité et coût au lancement.
2. **Flutter** pour l'app mobile → une base de code, perfs sur entrée de gamme.
3. **PostgreSQL + PostGIS** → relationnel + géospatial sans dépendance supplémentaire au début.
4. **Médias obligatoires + modération humaine** → garantie anti-arnaque, axe différenciant.
5. **Mobile Money en priorité** → adapté au marché cible.

Fin du document — version 1.0. À faire évoluer après validation des parties prenantes et cadrage juridique.